Bellamkonda Bhanu Prasad

Bath no :38

2303a52021

Task 1: Privacy and Data Security in AI-Generated Code

Scenario

AI tools can sometimes generate insecure authentication logic.

Task Description

Use an AI tool to generate a simple login system in Python.

Analyze the generated code to check:

• Whether credentials are hardcoded

• Whether passwords are stored or compared in plain text

• Whether insecure logic is used

Then, revise the code to improve security (e.g., avoid hardcoding, use input validation).

Expected Output

• AI-generated login code

• Identification of security risks

• Revised secure version of the code

• Brief explanation of improvements

CODE:

AIAC_LAB 5.3.py

AIAC_LAB 5.3.py > ...

```python
import hashlib

# Dictionary to store users {username: hashed_password}
users = {}

def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

def register():
    username = input("Enter new username: ").strip()
    if username in users:
        print("Username already exists. Please choose another.")
        return
    password = input("Enter new password: ").strip()
    if not username or not password:
        print("Invalid input. Username and password cannot be empty.")
        return
    users[username] = hash_password(password)
    print("Registration successful! Please login now.")

def login():
    username = input("Enter username: ").strip()
    password = input("Enter password: ").strip()
    if username in users and users[username] == hash_password(password):
        print("Login successful!")
    else:
        print("Login failed. Invalid username or password.")
```

```python

def main():
    while True:
        print("\n=== Welcome ===")
        print("1. Register")
        print("2. Login")
        print("3. Exit")
        choice = input("Choose an option: ").strip()

        if choice == "1":
            register()
        elif choice == "2":
            login()
        elif choice == "3":
            print("Goodbye!")
            break
        else:
            print("Invalid choice. Try again.")

main()
```

OUTPUT:

```
1. Register
2. Login
3. Exit
Choose an option: 1
Enter new username: bhanu
Enter new password: 12365
Registration successful! Please login now.

=== Welcome ===
1. Register
2. Login
3. Exit
Choose an option: 2
Enter username: bhanu
Enter password: 12365
Login successful!

=== Welcome ===
1. Register
2. Login
3. Exit
Choose an option: exit
Invalid choice. Try again.

=== Welcome ===
1. Register
2. Login
=== Welcome ===
1. Register
2. Login
1. Register
2. Login
2. Login
3. Exit
Choose an option: Exit
Choose an option: Exit
Invalid choice. Try again.
Invalid choice. Try again.
```


```
1. Register
2. Login
3. Exit
Choose an option: 1
Enter new username: bhanu
Enter new password: 12365
Registration successful! Please login now.

=== Welcome ===
1. Register
2. Login
3. Exit
Choose an option: 2
Enter username: bhanu
Enter password: 12365
Login successful!

=== Welcome ===
1. Register
2. Login
3. Exit
Choose an option: exit
Invalid choice. Try again.

=== Welcome ===
1. Register
2. Login
=== Welcome ===
1. Register
2. Login
1. Register
2. Login
2. Login
3. Exit
Choose an option: Exit
Choose an option: Exit
Invalid choice. Try again.
Invalid choice. Try again.
```

EXPLANATION:

We are creating a interface where user can login using the user and password .

Task 2: Bias Detection in AI-Generated Decision Systems

Scenario

AI systems may unintentionally introduce bias.

Task Description

Use AI prompts such as:

• “Create a loan approval system”

• Vary applicant names and genders in prompts

Analyze whether:

• The logic treats certain genders or names unfairly

• Approval decisions depend on irrelevant personal attributes

Suggest methods to reduce or remove bias.

Expected Output

• Python code generated by AI

• Identification of biased logic (if any)

• Discussion on fairness issues

• Mitigation strategies

CODE:

```
def loan_approval(name,gender,age,income,credit_score):
    if age < 18:
        return f"Sorry {name}, you are not eligible for a loan due to age restrictions."
    elif income < 20000:
        return f"Sorry {name}, you are not eligible for a loan due to insufficient income."
    elif credit_score < 600:
        return f"Sorry {name}, you are not eligible for a loan due to low credit score."
    else:
        return f"Congratulations {name}, your loan has been approved!"
# Example usage
name = input("Enter your name: ")
gender = input("Enter your gender: ")
age = int(input("Enter your age: "))
income = float(input("Enter your annual income: "))
credit_score = int(input("Enter your credit score: "))
result = loan_approval(name, gender, age, income, credit_score)
print(result)
```

OUTPUT:

```
PS C:\AI Assissted Coding Assessments> & C:/Python314/python.exe "c:/AI Assissted Coding Assessments/AIAC_LAB 5.3.py"
Enter your name: bhanu
Enter your gender: m
Enter your age: 19
Enter your annual income: 150000
Enter your credit score: 750
Congratulations bhanu, your loan has been approved!
PS C:\AI Assissted Coding Assessments>
```

We are checking the person can eligible for loan or not based on tha annual income and credit score.

Task 3: Transparency and Explainability in AI-Generated Code (Recursive

Binary Search)

Scenario

AI-generated code should be transparent, well-documented, and easy for

humans to understand and verify.

Task Description

Use an AI tool to generate a Python program that:

• Implements Binary Search using recursion

• Searches for a given element in a sorted list

• Includes:

o Clear inline comments

o A step-by-step explanation of the recursive logic

After generating the code, analyze:

• Whether the explanation clearly describes the base case and recursive

case

• Whether the comments correctly match the code logic

• Whether the code is understandable for beginner-level students

Expected Output

• Python program for recursive binary search

• AI-generated comments and explanation

• Student's assessment on clarity, correctness, and transparency

CODE:

```python
def binary_search(arr,target,low,high):
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] > target:
        return binary_search(arr, target, low, mid - 1)
    else:
        return binary_search(arr, target, mid + 1, high)
# Example usage
arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
target = int(input("Enter the number to search: "))
result = binary_search(arr, target, 0, len(arr) - 1)
if result != -1:
    print(f"Number found at index: {result}")
else:    print("Number not found in the array.")
```

OUTPUT:

```
Congratulations bhanu, your loan has been approved!
PS C:\AI Assissted Coding Assessments> & C:/Python314/python.exe "c:/AI Assissted Coding Assessments/AIAC_LAB 5.3.py"
Enter the number to search: 55
Number not found in the array.
PS C:\AI Assissted Coding Assessments>
```

EXPLANATION:

Here we are cunsturing the binary tree and searching then element in that tree .by using the function .

Task 4: Ethical Evaluation of AI-Based Scoring Systems

Scenario

AI-generated scoring systems can influence hiring decisions.

Task Description

Ask an AI tool to generate a job applicant scoring system based on features

such as:

• Skills

• Experience

• Education

Analyze the generated code to check:

• Whether gender, name, or unrelated features influence scoring

• Whether the logic is fair and objective

Expected Output

• Python scoring system code

• Identification of potential bias (if any)

• Ethical analysis of the scoring logic

CODE:

```python
def applicant_score(name,gender,skills,experience,education):
    score = 0
    if skills >= 5:
        score += 30
    elif skills >= 3:
        score += 20
    else:
        score += 10
    if experience >= 5:
        score += 30
    elif experience >= 3:
        score += 20
    else:
        score += 10
    if education.lower() == "bachelor":
        score += 20
    elif education.lower() == "master":
        score += 30
    else:
        score += 10
    return f"{name}, your applicant score is: {score}"
# Example usage
name = input("Enter your name: ")
gender = input("Enter your gender: ")
skills = int(input("Enter number of relevant skills: "))
experience = int(input("Enter years of experience: "))
education = input("Enter your highest education level (Bachelor/Master/Other): ")
result = applicant_score(name, gender, skills, experience, education)
print(result)
```

OUTPUT:

```
PS C:\AI Assissted Coding Assessments> & C:/Python314/python.exe "c:/AI Assissted Coding Assessments/AIAC_LAB 5.3.py"
Enter your name: BHANU
PS C:\AI Assissted Coding Assessments> & C:/Python314/python.exe "c:/AI Assissted Coding Assessments/AIAC_LAB 5.3.py"
Enter your name: BHANU
Enter your gender: M
Enter number of relevant skills: 3
Enter number of relevant skills: 3
Enter years of experience: 3
Enter years of experience: 3
Enter your highest education level (Bachelor/Master/Other): Bachelor
BHANU, your applicant score is: 60
PS C:\AI Assissted Coding Assessments>
```

EXPLANATION:

Here we are calculating the applicant score based on the skills,education,experience

Task 5: Inclusiveness and Ethical Variable Design

Scenario

Inclusive coding practices avoid assumptions related to gender, identity, or roles and promote fairness in software design.

Task Description

Use an AI tool to generate a Python code snippet that processes user or employee details.

Analyze the code to identify:

• Gender-specific variables (e.g., male, female)

• Assumptions based on gender or identity

• Non-inclusive naming or logic

Modify or regenerate the code to:

• Use gender-neutral variable names

• Avoid gender-based conditions unless strictly required

• Ensure inclusive and respectful coding practices

CODE:

```python
employees = [
    {"name": "John", "identity": "manager"},
    {"name": "Sarah", "identity": "developer"}
]

for emp in employees:
    print(f"{emp['name']} works as a {emp['identity']}")
```

OUTPUT:

```
BHANU, your applicant score is: 60
PS C:\AI Assissted Coding Assessments> & C:/Python314/python.exe "c:/AI Assissted Coding Assessments/AIAC_LAB 5.3.py"
John works as a manager
Sarah works as a developer
PS C:\AI Assissted Coding Assessments>
```

EXPLANATION:

Here we have an data of employee were tying to display the data of the employee using the loop.